Tugas Besar Machine Learning - Part 3

SmartClinic No-Show Predictor — Portasi model ke PyTorch dan penyimpanan model `.pth`

Nama: Yudi Ahmad Taqiyyudin | NPM: 202343501555 | Kelas: X7J Dosen: Ahmadi, S.Kom., M.T.I. | Mata kuliah: Machine Learning

Dataset: *Medical Appointment No Shows* (Kaggle, `KaggleV2-May-2016.csv`) — sama dengan Part 1 dan Part 2.

Alur: (1) Memuat data → (2) Preprocessing identik Part 2 → (3) Model MLP PyTorch → (4) Training (seed 42) → (5) Evaluasi + perbandingan dengan Part 2 → (6) Simpan `model_no_show.pth` → (7) Verifikasi muat ulang → (8) Download model.

Perbedaan Part 2 dan Part 3 (transparansi): Part 2 memakai tiga model scikit-learn (Logistic Regression, Decision Tree, Random Forest). Part 3 menambahkan model **PyTorch (MLP)** pada dataset, pembersihan data, pembagian data (80/20, stratified, `random_state=42`) dan preprocessing yang sama, agar model dapat disimpan dalam format `.pth` dan diintegrasikan ke aplikasi. Karena modelnya baru, seluruh metrik dievaluasi ulang.

0. Import Library

```
import os, json, random, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
                             confusion_matrix, ConfusionMatrixDisplay, roc_curve, average_precision_score)

warnings.filterwarnings("ignore")
RANDOM_STATE = 42

def set_seed(seed=RANDOM_STATE):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)

set_seed()
print("PyTorch:", torch.__version__, "| pandas:", pd.__version__)
```

```
PyTorch: 2.14.1+cpu | pandas: 2.2.3
```

1. Memuat Data

Unggah file `KaggleV2-May-2016.csv` ke Colab (atau letakkan di folder kerja).

```
PATH = "KaggleV2-May-2016.csv"

if not os.path.exists(PATH):
    try:
        from google.colab import files
        uploaded = files.upload()
        PATH = list(uploaded.keys())[0]
    except ImportError:
        raise FileNotFoundError(f"File {PATH} tidak ditemukan. Letakkan di folder yang sama dengan notebook.")

df = pd.read_csv(PATH)
print("Ukuran data:", df.shape)
df.head()
```

Choose Files KaggleV2-May-2016.csv

KaggleV2-May-2016.csv(text/csv) - 10739535 bytes, last modified: 11/6/2026 - 100% done
Saving KaggleV2-May-2016.csv to KaggleV2-May-2016.csv
Ukuran data: (110527, 14)

	PatientId	AppointmentID	Gender	ScheduledDay	AppointmentDay	Age	Neighbourhood	Scholarship	Hipertension	Diabetes	Alcoholis
0	2.987250e+13	5642903	F	2016-04-29T18:38:08Z	2016-04-29T00:00:00Z	62	JARDIM DA PENHA	0	1	0	
1	5.589978e+14	5642503	M	2016-04-29T16:08:27Z	2016-04-29T00:00:00Z	56	JARDIM DA PENHA	0	0	0	
2	4.262962e+12	5642549	F	2016-04-29T16:19:04Z	2016-04-29T00:00:00Z	62	MATA DA PRAIA	0	0	0	

2. Preprocessing (identik dengan Part 2)

- Nama kolom diseragamkan; `ScheduledDay` dan `AppointmentDay` diubah menjadi tanggal.
- Fitur baru: `waiting_days`, `appt_weekday` (0 = Senin), `sched_hour`.
- Baris tidak valid dibuang: `Age` di luar 0-100 dan `waiting_days` negatif.
- Target biner: `Yes` (tidak hadir) = 1, `No` (hadir) = 0. Kolom identitas dibuang.
- Pembagian **80% latih / 20% uji**, *stratified*, `random_state=42`.
- `StandardScaler` (numerik), `OneHotEncoder` dengan `min_frequency=100` (kategorik), biner apa adanya — hanya di-*fit* pada data latih.

```
NUM_COLS = ["Age", "waiting_days", "sched_hour", "Handicap"]
CAT_COLS = ["Gender", "Neighbourhood", "appt_weekday"]
BIN_COLS = ["Scholarship", "Hypertension", "Diabetes", "Alcoholism", "SMS_received"]
FEATURE_COLS = ["Gender", "Age", "Neighbourhood", "Scholarship", "Hypertension", "Diabetes",
                "Alcoholism", "Handicap", "SMS_received", "waiting_days", "appt_weekday", "sched_hour"]

data = df.rename(columns={"Hipertension": "Hypertension", "Handcap": "Handicap", "No-show": "No_show"})
data["ScheduledDay"] = pd.to_datetime(data["ScheduledDay"], utc=True).dt.tz_localize(None)
data["AppointmentDay"] = pd.to_datetime(data["AppointmentDay"], utc=True).dt.tz_localize(None)
data["waiting_days"] = (data["AppointmentDay"].dt.normalize() - data["ScheduledDay"].dt.normalize()).dt.days
data["appt_weekday"] = data["AppointmentDay"].dt.dayofweek.astype(str)
data["sched_hour"] = data["ScheduledDay"].dt.hour

before = len(data)
data = data[(data["Age"] >= 0) & (data["Age"] <= 100) & (data["waiting_days"] >= 0)]
print(f"Baris dihapus karena tidak valid: {before - len(data)}")

data["target"] = (data["No_show"] == "Yes").astype(int)
data = data.drop(columns=["PatientId", "AppointmentID", "ScheduledDay", "AppointmentDay", "No_show"])
data["Handicap"] = data["Handicap"].astype(int)

X = data.drop(columns="target")[FEATURE_COLS]
y = data["target"]
print("Ukuran data bersih:", data.shape, "| proporsi no-show: %.4f" % y.mean())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
print("Train:", X_train.shape, "| Test:", X_test.shape)

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), NUM_COLS),
    ("cat", OneHotEncoder(handle_unknown="infrequent_if_exist", min_frequency=100), CAT_COLS),
    ("bin", "passthrough", BIN_COLS),
])
```

```
Baris dihapus karena tidak valid: 13
Ukuran data bersih: (110514, 13) | proporsi no-show: 0.2019
Train: (88411, 12) | Test: (22103, 12)
```

2.1 Parameter preprocessing untuk model PyTorch

Parameter scaler dan encoder diekstrak dari `ColumnTransformer` dan disimpan sebagai angka/daftar sederhana di dalam file `.pth`, sehingga aplikasi tidak bergantung pada objek scikit-learn. Fungsi `transform` buatan sendiri diverifikasi **identik** dengan `ColumnTransformer`.

```
def extract_params(ct):
    scaler = ct.named_transformers_["num"]
    ohe = ct.named_transformers_["cat"]
    infreq = ohe.infrequent_categories_
```

```
    cats = {}
    for i, col in enumerate(CAT_COLS):
        all_c = [str(c) for c in ohe.categories_[i]]
        inf = [str(c) for c in infreq[i]] if infreq[i] is not None else []
        cats[col] = {"frequent": [c for c in all_c if c not in inf], "has_infrequent": len(inf) > 0}
    return {"num_cols": list(NUM_COLS), "cat_cols": list(CAT_COLS), "bin_cols": list(BIN_COLS),
            "num_mean": [float(v) for v in scaler.mean_], "num_scale": [float(v) for v in scaler.scale_],
            "categories": cats}

def feature_names(params):
    names = list(params["num_cols"])
    for col in params["cat_cols"]:
        info = params["categories"][col]
        names += [f"{col}={c}" for c in info["frequent"]]
        if info["has_infrequent"]:
            names.append(f"{col}=infrequent")
    return names + list(params["bin_cols"])

def transform(frame, params):
    n = len(frame)
    num = frame[params["num_cols"]].to_numpy(dtype=np.float64)
    num = (num - np.array(params["num_mean"])) / np.array(params["num_scale"])
    blocks = [num]
    for col in params["cat_cols"]:
        info = params["categories"][col]
        values = frame[col].astype(str).to_numpy()
        freq = info["frequent"]
        m = np.zeros((n, len(freq) + (1 if info["has_infrequent"] else 0)))
        for j, c in enumerate(freq):
            m[:, j] = values == c
        if info["has_infrequent"]:
            m[:, -1] = ~np.isin(values, freq)
        blocks.append(m)
    blocks.append(frame[params["bin_cols"]].to_numpy(dtype=np.float64))
    return np.hstack(blocks).astype(np.float32)

preprocessor.fit(X_train)
params = extract_params(preprocessor)
names = feature_names(params)
Xtr_all, Xte = transform(X_train, params), transform(X_test, params)

ref = preprocessor.transform(X_test)
ref = ref.toarray() if hasattr(ref, "toarray") else np.asarray(ref)
max_diff = float(np.abs(ref.astype(np.float32) - Xte).max())
print("Jumlah fitur setelah preprocessing:", Xtr_all.shape[1])
print("Selisih maksimum transform manual vs ColumnTransformer:", max_diff)
assert Xtr_all.shape[1] == len(names) and max_diff < 1e-4
```

```
Jumlah fitur setelah preprocessing: 92
Selisih maksimum transform manual vs ColumnTransformer: 0.0
```

3. Arsitektur Model (MLP PyTorch)

Multi-Layer Perceptron untuk klasifikasi biner: `Linear → BatchNorm → ReLU → Dropout` untuk setiap hidden layer, keluaran satu **logit**. Probabilitas *No-show* = `sigmoid(logit)`.

```
class NoShowMLP(nn.Module):
    def __init__(self, input_dim, hidden_dims=(64, 32), dropout=0.3):
        super().__init__()
        layers, prev = [], input_dim
        for h in hidden_dims:
            layers += [nn.Linear(prev, h), nn.BatchNorm1d(h), nn.ReLU(), nn.Dropout(dropout)]
            prev = h
        layers.append(nn.Linear(prev, 1))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x).squeeze(-1)

print(NoShowMLP(Xtr_all.shape[1], (64, 32), 0.3))
```

```
NoShowMLP(
  (net): Sequential(
    (0): Linear(in_features=92, out_features=64, bias=True)
    (1): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=64, out_features=32, bias=True)
    (5): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (6): ReLU()
```

```
    (7): Dropout(p=0.3, inplace=False)
    (8): Linear(in_features=32, out_features=1, bias=True)
  )
)
```

4. Training

- Validation set 10% diambil dari **data latih** (stratified) untuk *early stopping*; data uji hanya dipakai untuk evaluasi akhir.
- Ketidakseimbangan kelas: `BCEWithLogitsLoss(pos_weight = n_hadir / n_no-show)` (setara `class_weight="balanced"`).
- Konfigurasi terpilih dari pengujian sebelumnya (6 konfigurasi dibandingkan pada validation set, kriteria F1 seperti Part 2): hidden layers (64, 32), dropout 0,3, Adam, lr 1e-3, weight decay 1e-5, batch 512, maksimum 40 epoch, *patience* 6.
- Ubah `RUN_CONFIG_SEARCH = True` bila ingin mengulang perbandingan 6 konfigurasi (lebih lama).

```
BEST_CONFIG = {"hidden_dims": (64, 32), "dropout": 0.3, "lr": 1e-3, "weight_decay": 1e-5}
ALL_CONFIGS = [
    {"hidden_dims": (32,),     "dropout": 0.2, "lr": 1e-3, "weight_decay": 1e-5},
    BEST_CONFIG,
    {"hidden_dims": (64, 32),  "dropout": 0.3, "lr": 1e-3, "weight_decay": 1e-3},
    {"hidden_dims": (128, 64), "dropout": 0.4, "lr": 1e-3, "weight_decay": 1e-4},
    {"hidden_dims": (128, 64), "dropout": 0.5, "lr": 5e-4, "weight_decay": 1e-3},
    {"hidden_dims": (32, 16),  "dropout": 0.2, "lr": 1e-3, "weight_decay": 1e-4},
]
RUN_CONFIG_SEARCH = False
MAX_EPOCHS, PATIENCE, BATCH, THRESHOLD = 40, 6, 512, 0.5

def predict_proba(model, X_arr):
    model.eval()
    with torch.no_grad():
        return torch.sigmoid(model(torch.as_tensor(X_arr))).numpy()

def calc_metrics(y_true, prob, thr=THRESHOLD):
    pred = (prob >= thr).astype(int)
    return {"accuracy": float(accuracy_score(y_true, pred)),
            "precision": float(precision_score(y_true, pred, zero_division=0)),
            "recall": float(recall_score(y_true, pred)), "f1": float(f1_score(y_true, pred)),
            "roc_auc": float(roc_auc_score(y_true, prob)), "pr_auc": float(average_precision_score(y_true, prob)),
            "confusion_matrix": confusion_matrix(y_true, pred).tolist()}

def fit_one(cfg, Xtr, ytr, Xva, yva, pos_weight):
    set_seed()
    model = NoShowMLP(Xtr.shape[1], cfg["hidden_dims"], cfg["dropout"])
    opt = torch.optim.Adam(model.parameters(), lr=cfg["lr"], weight_decay=cfg["weight_decay"])
    loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(pos_weight, dtype=torch.float32))
    Xt, yt = torch.as_tensor(Xtr), torch.as_tensor(ytr, dtype=torch.float32)
    Xv, yv = torch.as_tensor(Xva), torch.as_tensor(yva, dtype=torch.float32)
    g = torch.Generator().manual_seed(RANDOM_STATE)
    best = {"val_loss": 1e9, "state": None, "epoch": 0}
    history, wait = [], 0
    for epoch in range(1, MAX_EPOCHS + 1):
        model.train()
        perm = torch.randperm(len(Xt), generator=g)
        tr_loss = 0.0
        for i in range(0, len(Xt), BATCH):
            idx = perm[i:i + BATCH]
            if len(idx) < 2:
                continue
            opt.zero_grad()
            loss = loss_fn(model(Xt[idx]), yt[idx])
            loss.backward(); opt.step()
            tr_loss += loss.item() * len(idx)
        model.eval()
        with torch.no_grad():
            val_loss = loss_fn(model(Xv), yv).item()
        history.append({"epoch": epoch, "train_loss": tr_loss / len(Xt), "val_loss": val_loss})
        if val_loss < best["val_loss"] - 1e-5:
            best = {"val_loss": val_loss, "state": {k: v.clone() for k, v in model.state_dict().items()}, "epoch": epoch}
            wait = 0
        else:
            wait += 1
            if wait >= PATIENCE:
                break
    model.load_state_dict(best["state"])
    return model, best["epoch"], history

ytr_all = y_train.to_numpy()
Xtr, Xva, ytr, yva = train_test_split(Xtr_all, ytr_all, test_size=0.1, stratify=ytr_all, random_state=RANDOM_STATE)
pos_weight = float((ytr == 0).sum() / (ytr == 1).sum())
print(f"pos_weight = {pos_weight:.3f} | fit {Xtr.shape[0]} | validasi {Xva.shape[0]}")
```

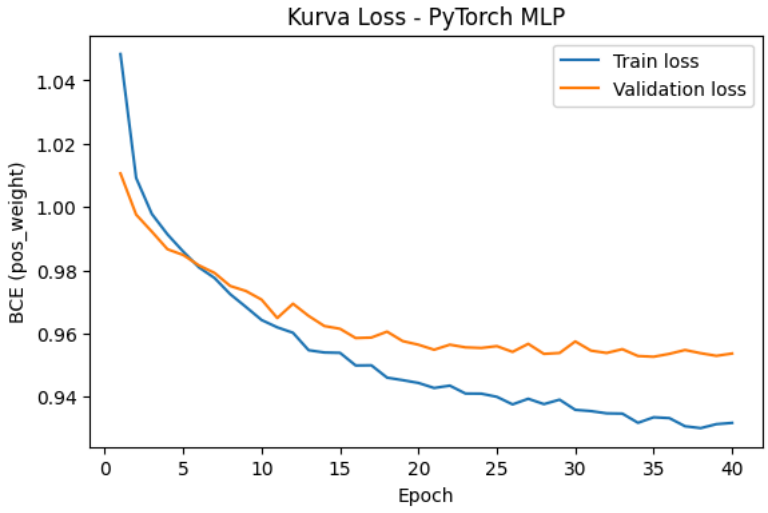
```
configs = ALL_CONFIGS if RUN_CONFIG_SEARCH else [BEST_CONFIG]
best_f1, search_rows = -1, []
for cfg in configs:
    m, ep, hist = fit_one(cfg, Xtr, ytr, Xva, yva, pos_weight)
    vm = calc_metrics(yva, predict_proba(m, Xva))
    search_rows.append({**{k: str(v) for k, v in cfg.items()}, "epoch_terbaik": ep,
                        "val_F1": round(vm["f1"], 4), "val_ROC_AUC": round(vm["roc_auc"], 4)})
    print(search_rows[-1])
    if vm["f1"] > best_f1:
        best_f1, best_cfg, best_model, best_hist, best_ep = vm["f1"], cfg, m, hist, ep
print("\nKonfigurasi dipakai:", best_cfg, "| epoch terbaik:", best_ep)
pd.DataFrame(search_rows)
```

```
pos_weight = 3.953 | fit 79569 | validasi 8842
{'hidden_dims': '(64, 32)', 'dropout': '0.3', 'lr': '0.001', 'weight_decay': '1e-05', 'epoch_terbaik': 35, 'val_F1': 0.4407, 'val_ROC_AUC': 0.728}

Konfigurasi dipakai: {'hidden_dims': (64, 32), 'dropout': 0.3, 'lr': 0.001, 'weight_decay': 1e-05} | epoch terbaik: 35
```

	hidden_dims	dropout	lr	weight_decay	epoch_terbaik	val_F1	val_ROC_AUC
0	(64, 32)	0.3	0.001	1e-05	35	0.4407	0.728

```
h = pd.DataFrame(best_hist)
plt.figure(figsize=(6.5, 4))
plt.plot(h.epoch, h.train_loss, label="Train loss"); plt.plot(h.epoch, h.val_loss, label="Validation loss")
plt.xlabel("Epoch"); plt.ylabel("BCE (pos_weight)"); plt.title("Kurva Loss - PyTorch MLP"); plt.legend(); plt.show()
```



5. Evaluasi pada Data Uji

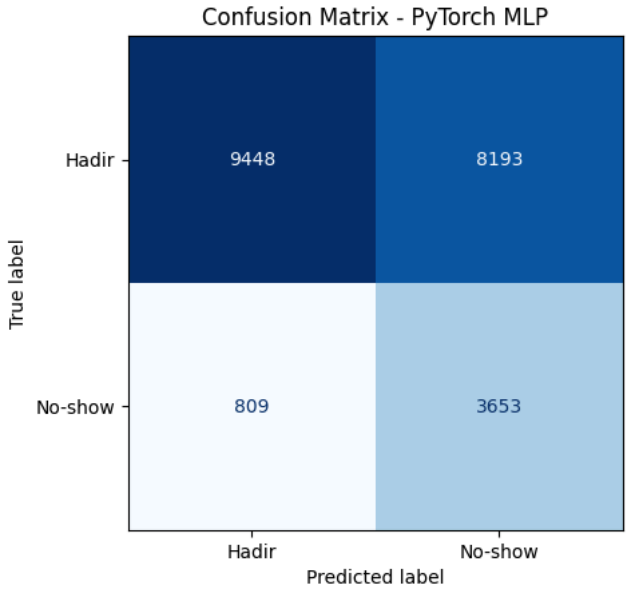
Metrik: Accuracy, Precision, Recall, F1-score, ROC-AUC dan Confusion Matrix (kelas positif = pasien **tidak hadir**, ambang 0,5). Diagnosis *good fit* memakai fungsi yang sama dengan Part 2 (selisih AUC latih-uji > 0,05 = overfit; AUC latih < 0,65 = underfit).

```
def diagnose(train_auc, test_auc, gap_thr=0.05, under_thr=0.65):
    if train_auc - test_auc > gap_thr:
        return "Overfit"
    if train_auc < under_thr:
        return "Underfit"
    return "Good fit"

p_tr = predict_proba(best_model, Xtr_all)
p_te = predict_proba(best_model, Xte)
res = calc_metrics(y_test.to_numpy(), p_te)
res["train_auc"] = float(roc_auc_score(ytr_all, p_tr))
res["gap_auc"] = res["train_auc"] - res["roc_auc"]
res["diagnosis"] = diagnose(res["train_auc"], res["roc_auc"])

print({k: (round(v, 4) if isinstance(v, float) else v) for k, v in res.items() if k != "confusion_matrix"})
ConfusionMatrixDisplay(np.array(res["confusion_matrix"]), display_labels=["Hadir", "No-show"]).plot(cmap="Blues", colorbar=False)
plt.title("Confusion Matrix - PyTorch MLP"); plt.show()
```

{'accuracy': 0.5927, 'precision': 0.3084, 'recall': 0.8187, 'f1': 0.448, 'roc_auc': 0.7339, 'pr_auc': 0.3654, 'train_auc': 0.7617, 'gap_auc': 0.0278, 'diagnosis': 'Good fit'}



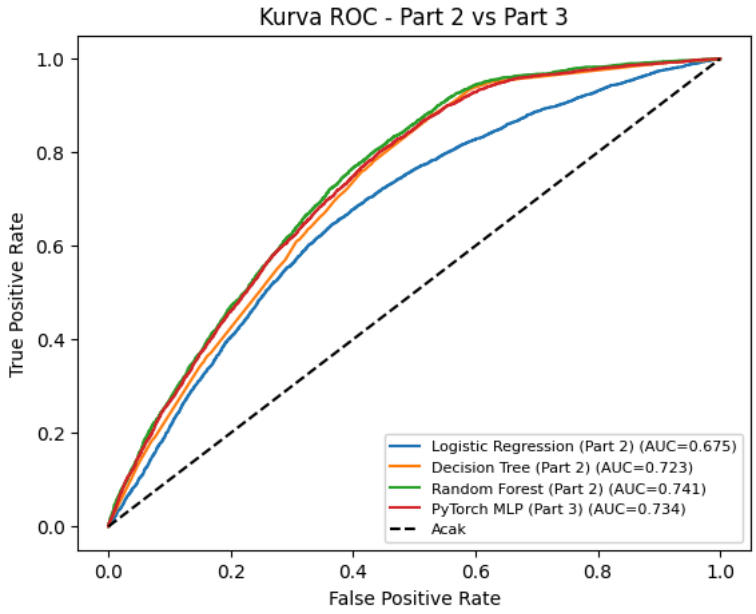
5.1 Perbandingan dengan Part 2

Model scikit-learn dilatih ulang dengan **hyperparameter terbaik hasil tuning Part 2** (direproduksi pada pipeline dan pembagian data yang sama), sehingga perbandingan dilakukan pada data uji yang sama.

```
sk_models = {
    "Logistic Regression": LogisticRegression(C=0.001, class_weight="balanced", max_iter=1000, random_state=RANDOM_STATE),
    "Decision Tree": DecisionTreeClassifier(max_depth=7, min_samples_leaf=200, criterion="entropy",
                                            class_weight="balanced", random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(n_estimators=200, max_depth=12, min_samples_leaf=5, max_features=0.5,
                                            class_weight="balanced", random_state=RANDOM_STATE, n_jobs=-1),
}
probs = {}
rows = []
for name, est in sk_models.items():
    pipe = Pipeline([("prep", preprocessor), ("clf", est)]).fit(X_train, y_train)
    pte, ptr = pipe.predict_proba(X_test)[:, 1], pipe.predict_proba(X_train)[:, 1]
    probs[f"{name} (Part 2)"] = pte
    mm = calc_metrics(y_test.to_numpy(), pte)
    ta = float(roc_auc_score(y_train, ptr))
    rows.append({"Model": f"{name} (Part 2)", "Accuracy": mm["accuracy"], "Precision": mm["precision"],
                 "Recall": mm["recall"], "F1": mm["f1"], "ROC-AUC": mm["roc_auc"],
                 "Gap AUC": ta - mm["roc_auc"], "Diagnosis": diagnose(ta, mm["roc_auc"])})
probs["PyTorch MLP (Part 3)"] = p_te
rows.append({"Model": "PyTorch MLP (Part 3)", "Accuracy": res["accuracy"], "Precision": res["precision"],
             "Recall": res["recall"], "F1": res["f1"], "ROC-AUC": res["roc_auc"],
             "Gap AUC": res["gap_auc"], "Diagnosis": res["diagnosis"]})
comparison = pd.DataFrame(rows).set_index("Model").round(4)
display(comparison)

plt.figure(figsize=(6.5, 5))
for name, p in probs.items():
    fpr, tpr, _ = roc_curve(y_test, p)
    plt.plot(fpr, tpr, label=f"{name} (AUC={roc_auc_score(y_test, p):.3f})")
plt.plot([0, 1], [0, 1], "k--", label="Acak")
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate"); plt.title("Kurva ROC - Part 2 vs Part 3")
plt.legend(fontsize=8); plt.show()
```

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Gap AUC	Diagnosis
Logistic Regression (Part 2)	0.6673	0.3201	0.5764	0.4116	0.6749	-0.0008	Good fit
Decision Tree (Part 2)	0.5632	0.2983	0.8606	0.4430	0.7233	0.0102	Good fit
Random Forest (Part 2)	0.6181	0.3199	0.7925	0.4558	0.7414	0.0559	Overfit
PyTorch MLP (Part 3)	0.5927	0.3084	0.8187	0.4480	0.7339	0.0278	Good fit



Next steps: Generate code with comparison | New interactive sheet

6. Simpan Model .pth

Checkpoint berisi bobot model (`state_dict`), konfigurasi arsitektur, parameter preprocessing, urutan fitur, pemetaan kelas, ambang keputusan dan metrik uji — semua yang dibutuhkan aplikasi Streamlit. Model **dilatih sebagai model PyTorch**, bukan hasil mengganti ekstensi file scikit-learn.

```
MODEL_FILE = "model_no_show.pth"
checkpoint = {
    "model_state_dict": best_model.state_dict(),
    "model_config": {"input_dim": int(Xtr.shape[1]), "hidden_dims": list(best_cfg["hidden_dims"]),
                     "dropout": float(best_cfg["dropout"])},
    "preprocessing": params,
    "feature_columns": FEATURE_COLS,
    "feature_names_out": names,
    "class_mapping": {"0": "Hadir", "1": "Tidak hadir (No-show)"},
    "threshold": THRESHOLD,
    "metrics_test": {k: v for k, v in res.items() if k != "confusion_matrix"},
    "training_info": {"framework": f"PyTorch {torch.__version__}", "random_state": RANDOM_STATE,
                      "dataset": "Medical Appointment No Shows (Kaggle)", "best_epoch": int(best_ep),
                      "pos_weight": pos_weight, "batch_size": BATCH, "optimizer": "Adam"},
}
torch.save(checkpoint, MODEL_FILE)
print(f"Model tersimpan: {os.path.abspath(MODEL_FILE)}")
print(f"Ukuran file   : {os.path.getsize(MODEL_FILE) / 1024:.1f} KB")
```

```
Model tersimpan: /content/model_no_show.pth
Ukuran file   : 44.4 KB
```

7. Verifikasi Model Dapat Dimuat Ulang

```
ck = torch.load(MODEL_FILE, map_location="cpu", weights_only=True)
cfg = ck["model_config"]
loaded = NoShowMLP(cfg["input_dim"], tuple(cfg["hidden_dims"]), cfg["dropout"])
loaded.load_state_dict(ck["model_state_dict"])
loaded.eval()

p_loaded = predict_proba(loaded, transform(X_test, ck["preprocessing"]))
auc_re = roc_auc_score(y_test, p_loaded)
```

```
f1_re = f1_score(y_test, (p_loaded >= ck["threshold"]).astype(int))

print("Kunci checkpoint :", list(ck.keys()))
print("Konfigurasi      :", cfg)
print("Pemetaan kelas   :", ck["class_mapping"], "| threshold:", ck["threshold"])
print(f"Selisih prediksi model asli vs hasil muat ulang (maks): {np.abs(p_loaded - p_te).max():.2e}")
print(f"ROC-AUC uji dari .pth: {auc_re:.4f} | F1 uji dari .pth: {f1_re:.4f}")
assert np.abs(p_loaded - p_te).max() < 1e-6, "Prediksi tidak konsisten!"
assert abs(auc_re - ck["metrics_test"]["roc_auc"]) < 1e-6
print("VERIFIKASI BERHASIL: model dapat dimuat ulang dan menghasilkan prediksi yang konsisten.")
```

```
Kunci checkpoint : ['model_state_dict', 'model_config', 'preprocessing', 'feature_columns', 'feature_names_out', 'class_mapping', 'threshold', 'metrics_test', 'training_info']
Konfigurasi      : {'input_dim': 92, 'hidden_dims': [64, 32], 'dropout': 0.3}
Pemetaan kelas   : {'0': 'Hadir', '1': 'Tidak hadir (No-show)'} | threshold: 0.5
Selisih prediksi model asli vs hasil muat ulang (maks): 0.00e+00
ROC-AUC uji dari .pth: 0.7339 | F1 uji dari .pth: 0.4480
VERIFIKASI BERHASIL: model dapat dimuat ulang dan menghasilkan prediksi yang konsisten.
```

8. Download Model dari Colab